Divide and Conquer

Jakarin Chawachat

Agenda

- เกริ่นนำ
- Sorting return
  - Merge sort
  - Quick sort

Divide and Conquer (D&C)

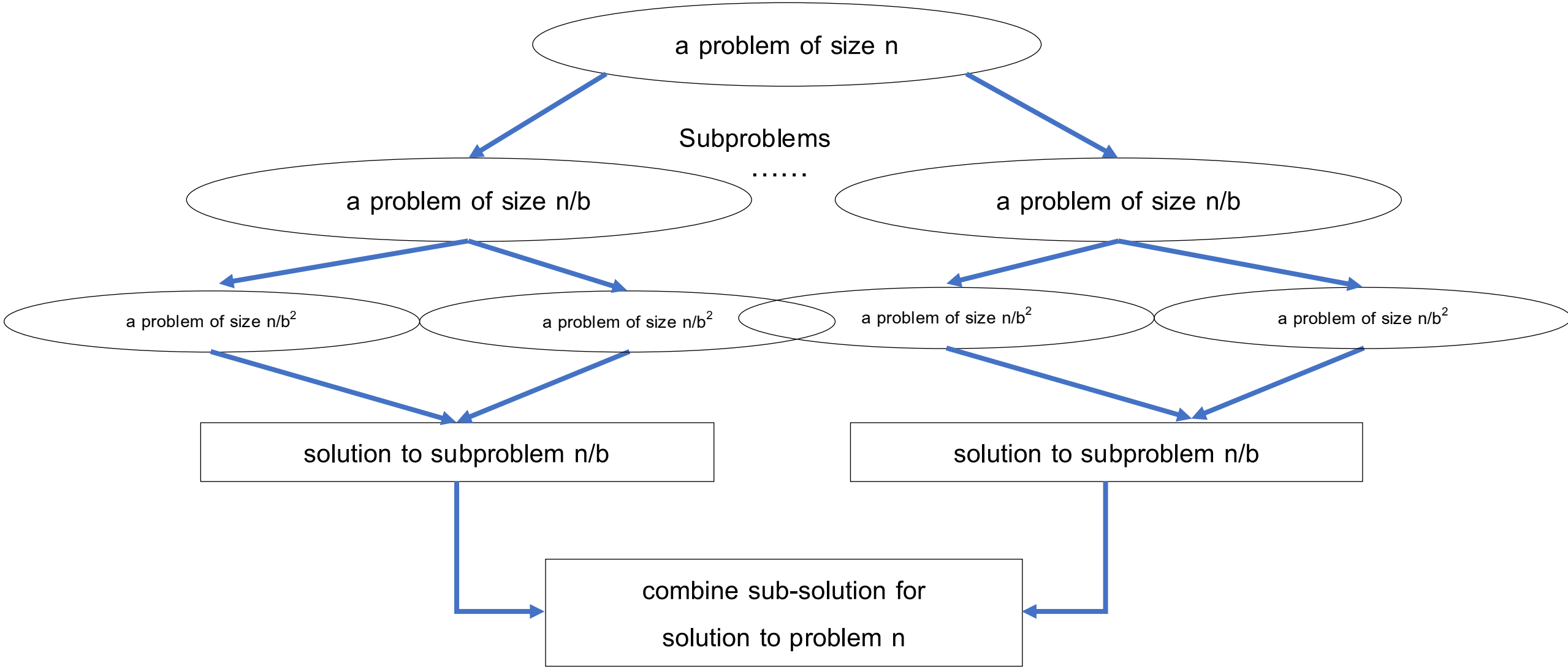
- เป็นรูปแบบการแก้ปัญหาที่ปัญหาจะถูกทำให้ง่ายลง โดยการแบ่งเป็นส่วนย่อยๆ จากนั้นทำการจัดการแต่ละส่วน

- ขั้นตอนโดยทั่วไป
  - แบ่งปัญหาตั้งต้นเป็นปัญหาย่อย subproblems (โดยทั่วไปแบ่งครึ่ง หรือเกือบ)
  - หาคำตอบย่อย (subsolution) ของปัญหาย่อยเหล่านั้น (ซึ่งง่ายขึ้นแล้ว)
  - **ถ้าจำเป็น** รวมคำตอบย่อย เพื่อให้เป็นคำตอบที่สมบูรณ์

Divide and Conquer Visualization

- ส่วนใหญ่เราก็สมมติว่า ปัญหามีขนาด n
- เราจะแบ่งเป็นปัญหาย่อยๆ ที่แต่ละอันมีขนาด n/b เมื่อ b เป็นค่าคงที่ เช่น n/2
- เราจะแก้ปัญหาย่อยแบบ **recursive** แล้วรวมคำตอบกลับมาเป็นปัญหาหลัก

Divide and Conquer Visualization



Divide and Conquer Visualization

```
DivideAndConquer(P) {
        if(small(P)) //P is very small so that a solution obvious
                return solution(n);
        divide the problem P into k subproblems P1, P2, … ,Pk :
        return (Combine(DivideAndConquer(P1),
            DivideAndConquer(P2),…,  DivideAndConquer(PK)))
}
```

ตัวอย่างที่เห็นได้ชัด

Binary search (บางคนบอกว่าเป็น decrease and conquer)

- สมมติว่าเรามีเลข n ตัวที่เรียงแล้วจากน้อยไปมาก แล้วต้องการหา X ว่าอยู่ในเลขชุดนี้ไหม (เหมือนเกมทายเลขที่เราเคยเล่นกัน) ทำอย่างไรดี

- เราก็เลือกทายตำแหน่งไหนก่อน.....
- เราก็ตรวจสอบตรงกลาง array ก่อนว่ามีของที่เราต้องการไหม

- ถ้าไม่เช่นนั้นเราก็จะตัดสินใจว่า คำตอบอยู่ทางด้านซ้าย / ขวาของตัวกึ่งกลางและทำการค้นหาต่อ

- การที่เราพิจารณาปัญหาที่มีขนาดลดลงครึ่งหนึ่งทำให้ ได้ O(logn)

- ลองทำ binary search ใน elab
- ลองเขียนด้วย recursive
- https://cmu.to/binarysearch

เฉลย binary search

```
int binarySearch(int A[], int key, int l, int r)
{
    if (r >= l) {
        int mid = l + (r - l) / 2;
        if (A[mid] == key)
            return mid;
        if (A[mid] > key)
            return binarySearch(A, key, l, mid - 1);
        return binarySearch(A, key, mid + 1, r);
    }
    return -1;
}
```

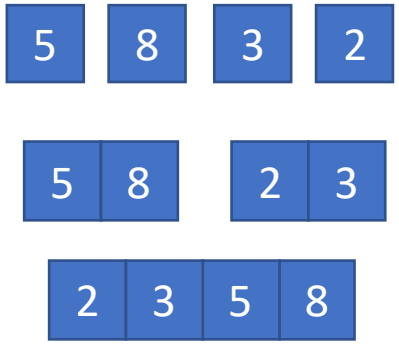
กลับมา Sorting

- ก่อนหน้านี้เราเรียน Basic sorting algorithms ไปแล้ว
  - Selection sort
  - Bubble sort
  - Insertion sort

- วันนี้เราจะมีเรียน Sorting algorithms 2 อันที่ใช้หลักการ D&C
  - Merge sort
  - Quick sort

Merge sort

- กำหนด array ของ N จำนวนมาให้ merge sort จะทำดังนี้
  - แต่ละคู่ของ array ที่เรียงแล้วขนาด 1 จำนวน รวมกัน เป็นขนาด 2 จำนวน
  - แต่ละคู่ของ array ที่เรียงแล้วขนาด 2 จำนวน รวมกัน เป็นขนาด 4 จำนวน
  - ทำลักษณะนี้ต่อไปเรื่อยๆ
  - สุดท้ายนำ array ที่เรียงแล้วขนาด n/2 ซึ่งมี 2 อัน มา รวมกัน เป็นขนาด n จำนวนที่เรียงแล้ว



มา Merge กันก่อนนน

- ก่อนอื่นเรามาดูวิธีการรวม array ที่เรียงแล้ว 2 อันกันก่อน

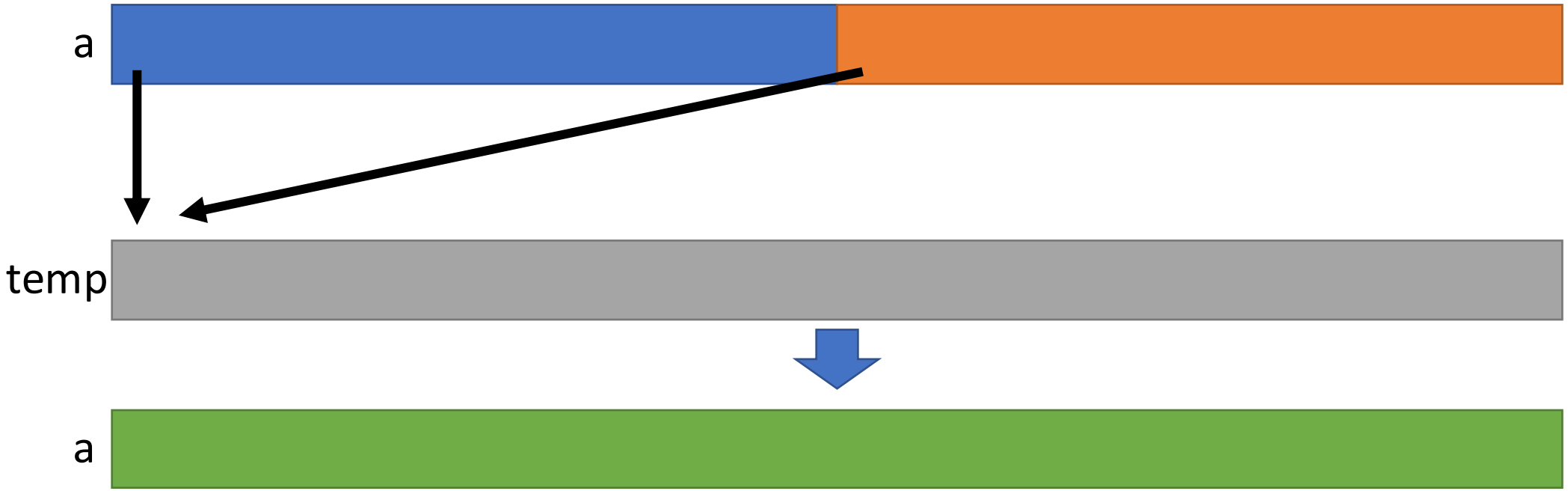
- ถ้าเรามี array A และ B ที่เรียงแล้ว เราจะทำอย่างไรเพื่อให้ได้ array ที่เรียงแล้วของ AUB เป็นวิธีที่ง่ายๆ

มา Merge กันก่อนนน

- ก่อนอื่นเรามาดูวิธีการรวม array ที่เรียงแล้ว 2 อันกันก่อน
- ถ้าเรามี array A และ B ที่เรียงแล้ว เราจะทำอย่างไรเพื่อให้ได้ array ที่เรียงแล้วของ AUB เป็นวิธีที่ง่ายๆ
- เราก็พิจารณาตัวหน้าสุดของแต่ละ array ไง ถ้าใครน้อยกว่าก็เอาใส่ array temp ไว้ก่อน

- ลองเขียน Merge array ดูกัน แต่เพิ่มความยาก สมมติว่ามี array เดียว แต่แบ่งเป็น 2 ส่วนตรงกลาง นั่นคือ
  - ตัวแรกของ array1 อยู่ที่ 0
  - ส่วนตัวแรกของ array2 อยู่ที่ mid
  - เปรียบเทียบแล้วเอาใส่ array temp ไว้ก่อน แล้วค่อยเอามาทับ array เดิม

void merge(int a[], int low, int mid, int high) {…}



ลองเขียน merge ใน elab ดู

https://cmu.to/mergepart

เฉลย Merge

```
void merge(int a[], int low, int mid, int high) {
  int N = high-low+1, b[N]; // discuss: why do we need a temporary array b?
  int left = low, right = mid+1, bIdx = 0;
  while (left <= mid && right <= high){ // the merging
    if (a[left] <= a[right]){
      b[bIdx++] = a[left++];
    }else{
      b[bIdx++] = a[right++];
    }
  }
  while (left <= mid){
    b[bIdx++] = a[left++]; // leftover, if any
  }
  while (right <= high){
    b[bIdx++] = a[right++]; // leftover, if any
  }
  for (int k = 0; k < N; k++)
    a[low+k] = b[k]; // copy back
}
```

ทีนี้ Divide and conquer ยังไง

- ในปัญหานี้ เราทำการแบ่งแยกและเอาชนะตามขั้นตอนนี้

- ขั้นตอนการแบ่ง
  - เราแบ่งปัญหาใหญ่ เริ่มต้น เป็นปัญหาย่อยที่เล็กลงและ recursive แก้ปัญหาย่อย

- ขั้นตอนการเอาชนะ
  - รวมคำตอบของปัญหาย่อยเพื่อสร้างเป็นผลลัพธ์ของปัญหาใหญ่ เริ่มต้น

อธิบายเพิ่มเติม

ใน Merge sort นั้น

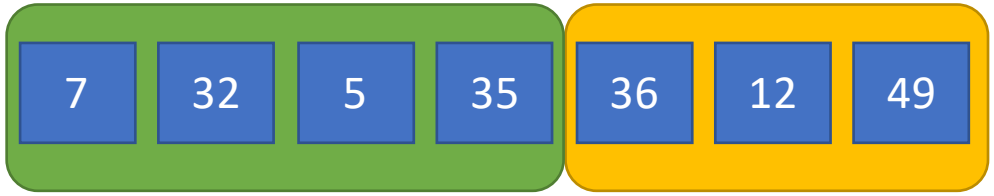
- ขั้นตอนการแบ่ง ง่าย ก็คือ เราแบ่ง array ปัจจุบันเป็น 2 ส่วน และ recursive เรียก sort ทั้งสองส่วน (แบ่งจนเหลือกี่ตัว)

- ขั้นตอนการเอาชนะ รวม array ที่เรียงแล้ว 2 ส่วนเป็น array ที่เรียงแล้วที่ใหญ่ขึ้น (ที่เราเขียนไปก่อนนี้แล้วไง)

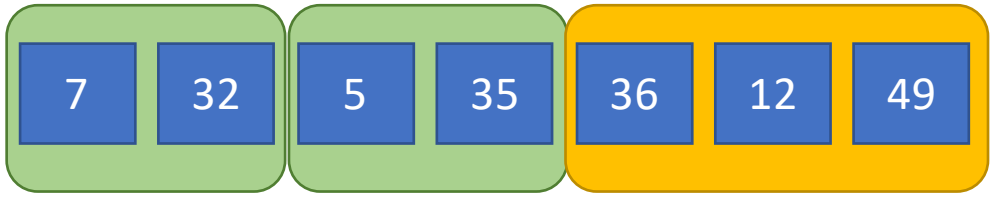
```
void mergeSort(int a[], int low, int high) {
  // the array to be sorted is a[low..high]
  if (low < high) { // base case: low >= high (0 or 1 item)
    int mid = (low+high) / 2;
    mergeSort(a, low  , mid ); // divide into two halves
    mergeSort(a, mid+1, high); // then recursively sort them
    merge(a, low, mid, high); // conquer: the merge subroutine
  }
}
```

เนื่องจากเป็น recursive มันจะเรียกกลุ่มด้านซ้าย ทำให้เสร็จจึงเรียกกลุ่มด้านขวา
ดังนั้นในการทำงาน มันจะเรียกกลุ่มด้านซ้าย ด้านซ้าย ด้านซ้าย จนเหลือ 0 หรือ 1 ตัว
แล้วค่อยเรียกกลุ่มด้านขวาแล้วค่อยรวม

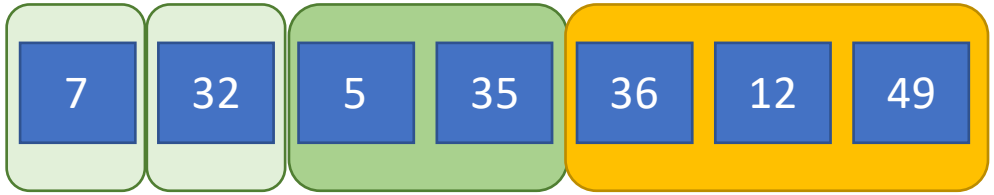
เริ่มต้น



เรียก mergeSort ได้ก้อน 1 ถ้าทำเสร็จเรียกก้อน 2
แล้วค่อย merge

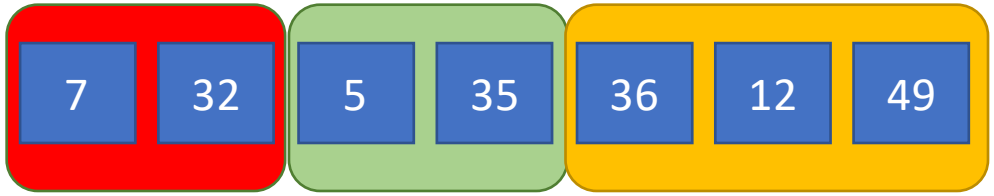


ส่วนของก้อน 1 เรียก mergeSort ได้
ก้อน 1.1 ถ้าทำเสร็จเรียกก้อน 1.2
แล้วค่อย merge

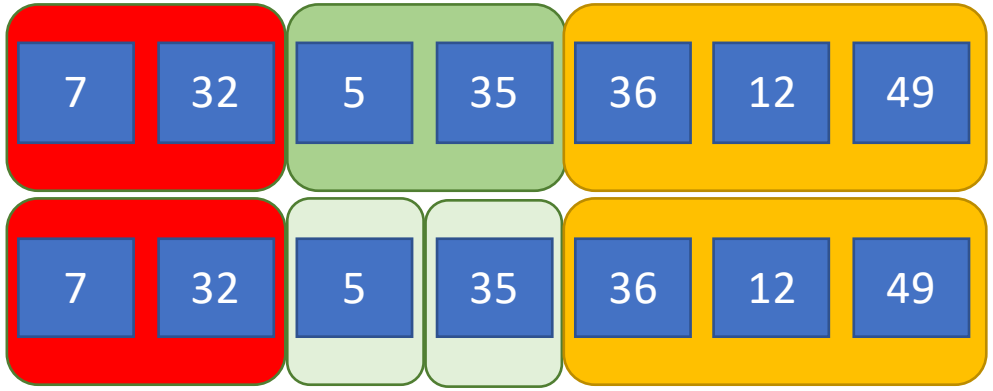


ส่วนของก้อน 1.1 เรียก mergeSort ได้
ก้อน 1.1.1 แต่มีตัวเดียว เรียกแล้ว return
ก้อน 1.1.2 ก็เช่นเดียวกัน เรียกแล้ว return

ดังนั้น merge ก้อน 1.1.1 และ 1.1.2

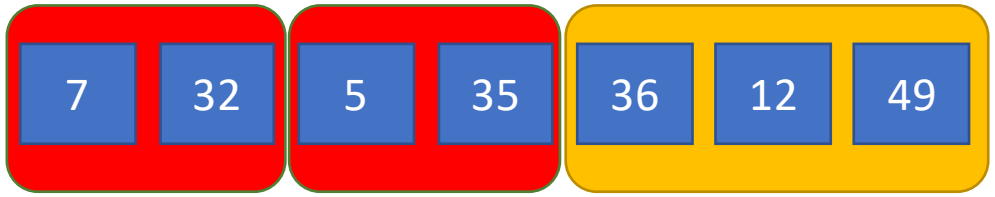


ทำให้ได้ก้อน 1.1 ที่เรียงแล้ว



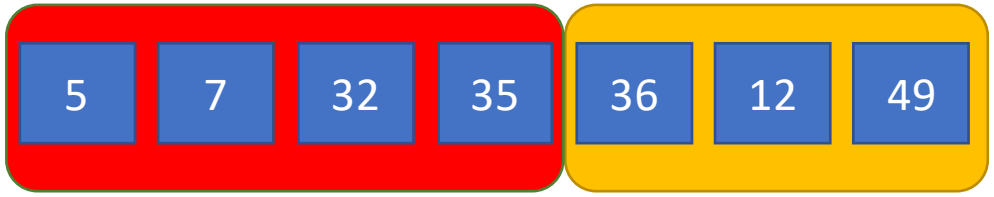
ส่วนของก้อน 1.2 เรียก mergeSort ได้
ก้อน 1.2.1 แต่มีตัวเดียว เรียกแล้ว return
ก้อน 1.2.2 ก็เช่นเดียวกัน เรียกแล้ว return

ดังนั้น merge ก้อน 1.2.1 และ 1.2.2

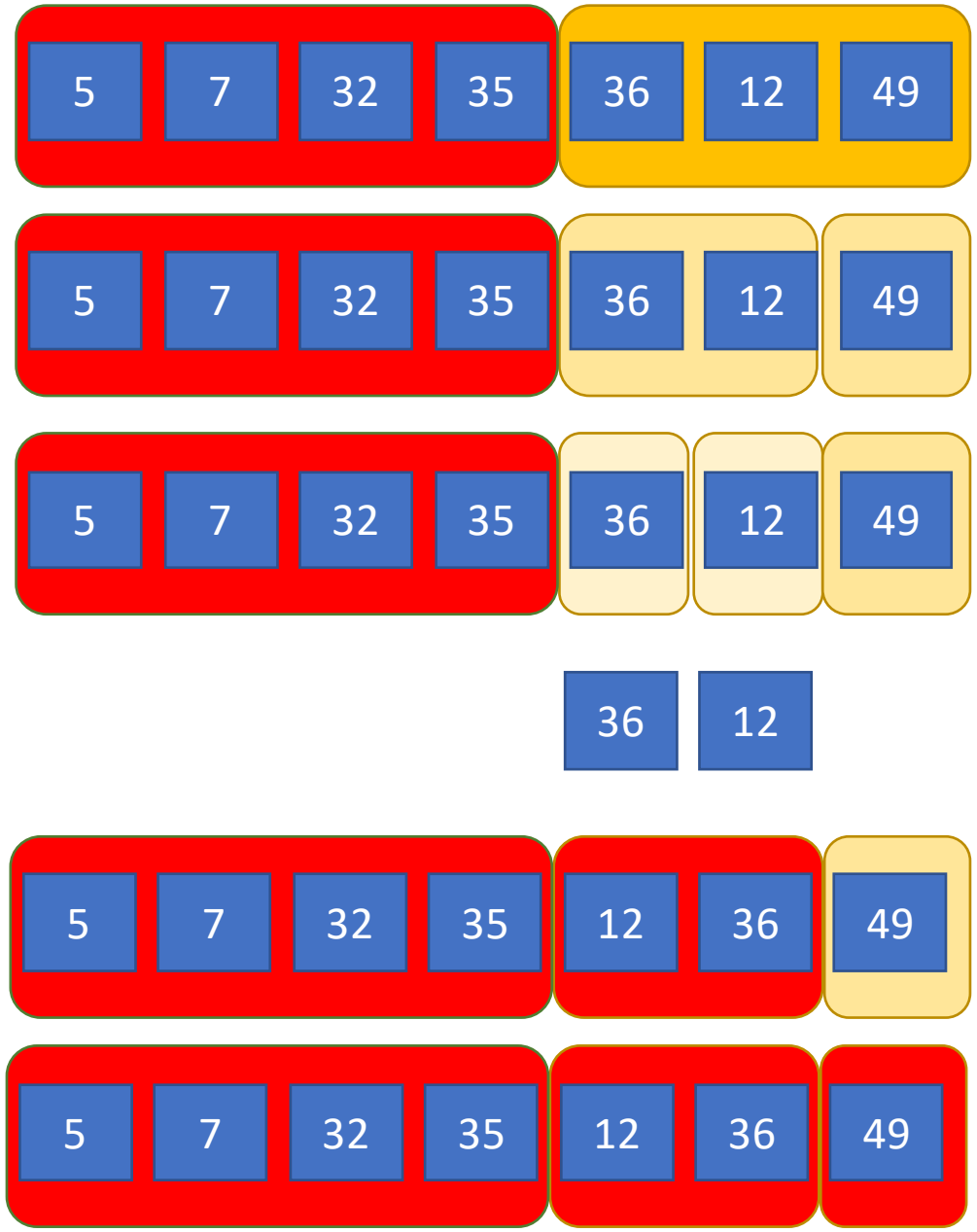


ทำให้ได้ก้อน 1.2 ที่เรียงแล้ว

ดังนั้น merge ก้อน 1.1 และ 1.2



ทำให้ได้ก้อน 1 ที่เรียงแล้ว



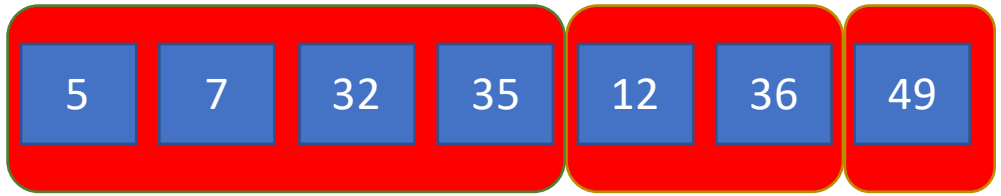
ส่วนของก้อน 2 เรียก mergeSort ได้
ก้อน 2.1 ถ้าทำเสร็จเรียกก้อน 2.2
แล้วค่อย merge

ส่วนของก้อน 2.1 เรียก mergeSort ได้
ก้อน 2.1.1 แต่มีตัวเดียว เรียกแล้ว return
ก้อน 2.1.2 ก็เช่นเดียวกัน เรียกแล้ว return

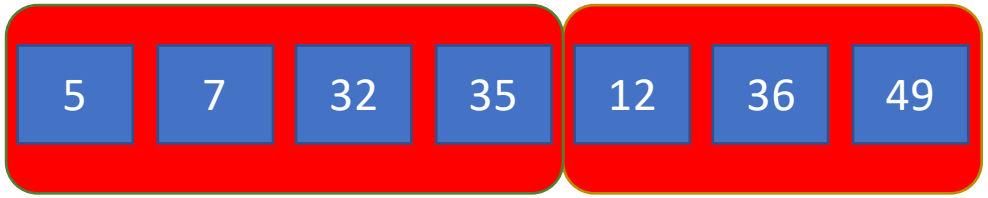
ดังนั้น merge ก้อน 2.1.1 และ 2.1.2

ทำให้ได้ก้อน 2.1 ที่เรียงแล้ว

ก้อน 2.2 มีตัวเดียว เรียงแล้ว

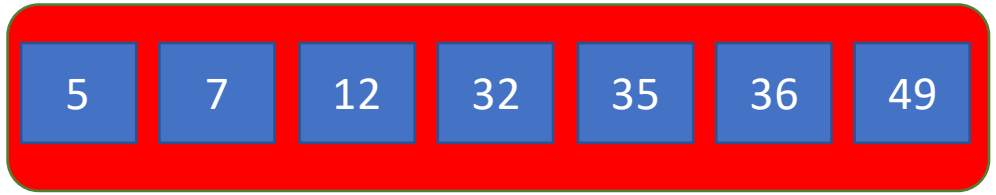


ดังนั้น merge ก้อน 2.1 และ 2.2



ทำให้ได้ก้อน 2 ที่เรียงแล้ว

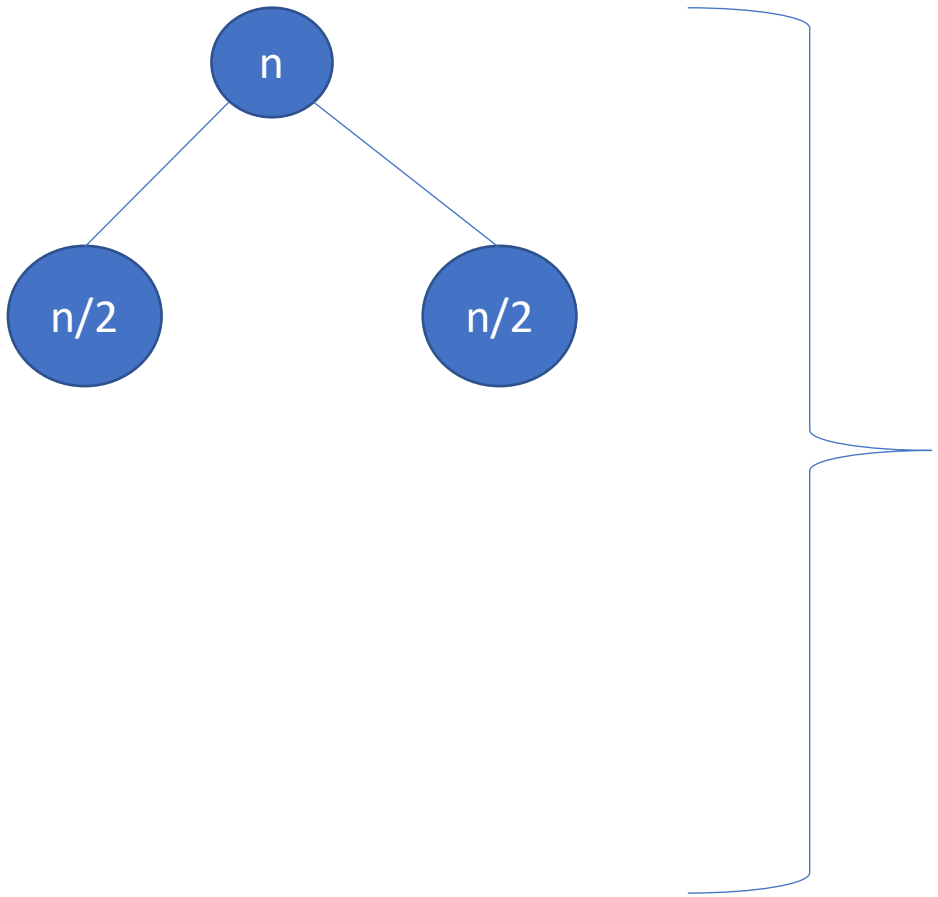
ดังนั้น merge ก้อน 1 และ 2



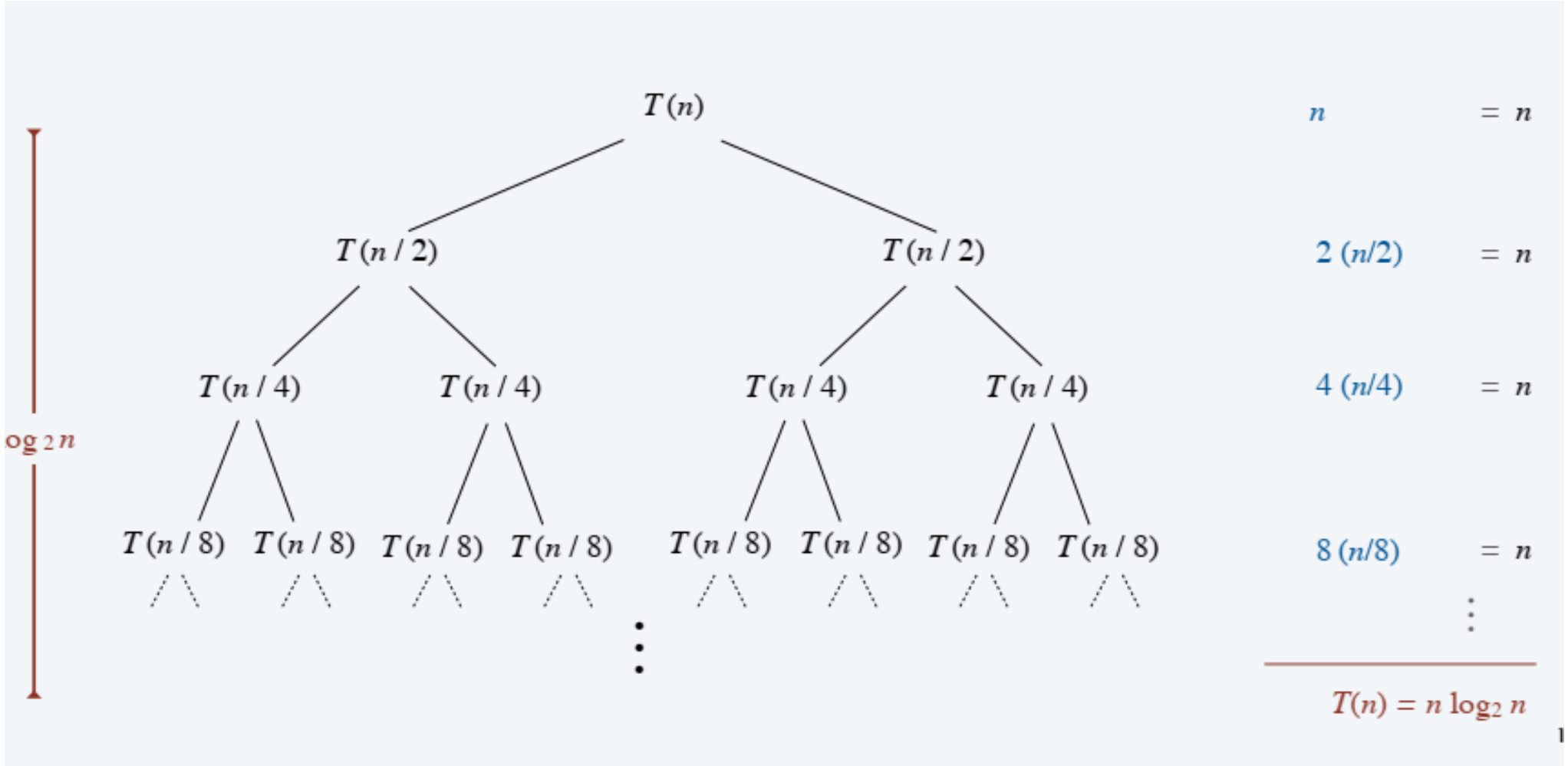
ทำให้ได้ข้อมูลที่เรียงแล้ว

วิเคราะห์เวลาในการทำงาน

- $T(n) = \begin{cases} 1 \\ 2T\left(\frac{n}{2}\right) + n \end{cases}$




```
void mergeSort(int a[], int low, int high) {
  // the array to be sorted is a[low..high]
  if (low < high) {
    int mid = (low+high) / 2;
    mergeSort(a, low  , mid );
    mergeSort(a, mid+1, high);
    merge(a, low, mid, high);
  }
}
```


T(n)
T(n / 2)
T(n / 2)
T(n / 4)
T(n / 4)
T(n / 4)
T(n / 4)
T(n / 8)
T(n / 8)
T(n / 8)
T(n / 8)
T(n / 8)
T(n / 8)
T(n / 8)
T(n / 8)
og 2 n
n = n
2 (n/2) = n
4 (n/4) = n
8 (n/8) = n
T(n) = n log2 n

สรุป Merge sort

ข้อดี

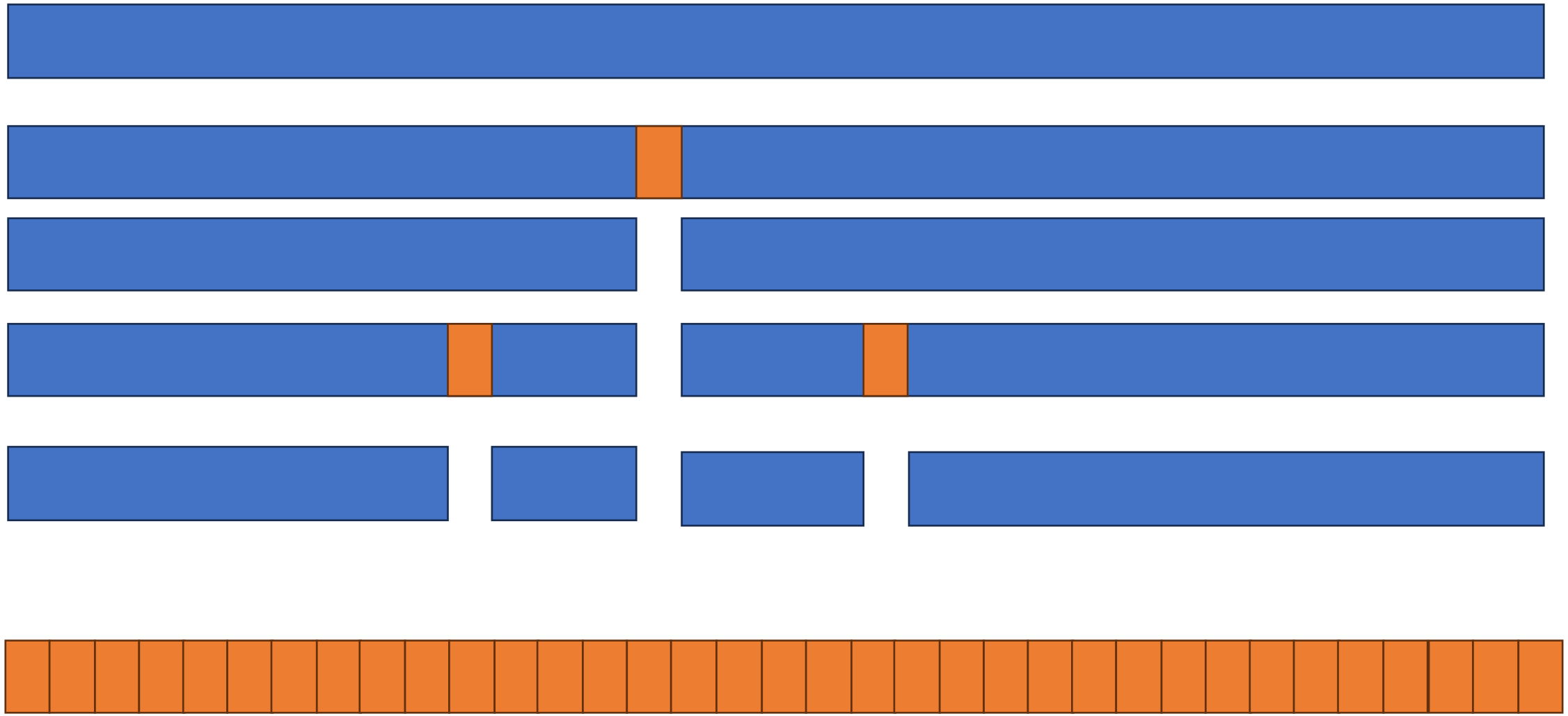
- ทำงานใน O(NlogN)
- เหมาะกับเมื่อมีเลขจำนวนมากเพราะว่า Basic sorting algorithmใช้$O(n^2)$

ข้อเสีย

- implement ยากหน่อย
- ต้องใช้พื้นที่สำรองตอน merge อีก O(N)

Quick sort

- เป็นอัลกอริทึม Divide and conquer อีกตัว
- ขั้นตอนการแบ่ง
  - เลือกข้อมูลใน array a[i..j] มาตัวหนึ่ง สมมติว่าเป็น p (ส่วนใหญ่เรียกว่า pivot)
  - แบ่งข้อมูล a[i..j] ออกเป็น 3 ส่วน a[i..m-1], a[m], a[m+1..j]
    - a[i..m-1] คือกลุ่มของสมาชิกที่น้อยกว่า p (อาจจะไม่มีสักตัวก็ได้นะ)
    - a[m] คือ pivot p นั่นคือ index m คือตำแหน่งที่ถูกต้องของ p ใน array ที่เรียงแล้ว
    - a[m+1..j] คือกลุ่มของสมาชิกที่มากกว่าหรือเท่ากับ p
  - Recursive ทำ 2 ส่วนต่อ (ส่วนไหน)
- ขั้นตอนการเอาชนะ
  - โอ้ลา ไม่ต้องทำอะไร ทำไมหละ ๆ



```
void quickSort(int a[], int low, int high) {
  if (low < high) {
    int m = partition(a, low, high); // O(N)
    quickSort(a, low, m-1); // recursively sort left subarray
    quickSort(a, m+1, high); // then sort right subarray
  }
}
```

มาแบ่งกันก่อนนนน

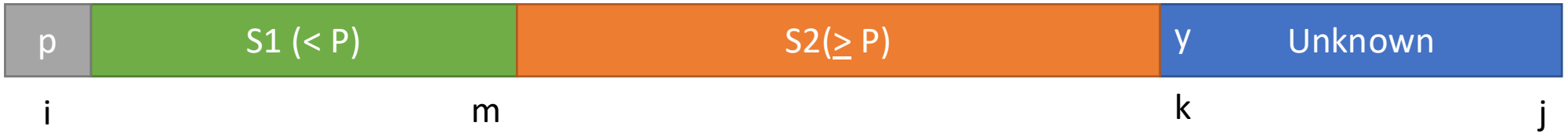
- เช่นเดียวกับตอนดู Merge sort ที่เราดู Merge ทีนี้เราก็จะมาดู ฟังก์ชันที่สำคัญ นั่นคือ partition (แบ่ง)
- ในการแบ่ง a[i..j] นั้น เริ่มต้นง่ายสุด เราก็เลือก a[i] มาเป็น pivot
- สมาชิกที่เหลือ a[i+1..j] จะถูกแบ่งเป็น 3 ส่วน
  - ส่วน S1 = a[i+1..m] เมื่อแต่ละสมาชิกนั้น $\leq$ p
  - ส่วน S2 = a[m+1..k-1] เมื่อแต่ละสมาชิกนั้น > p
  - ส่วน Unknown = a[k..j] คือส่วนที่ยังไม่ได้ดูซึ่งถ้าดูแล้วก็จะไปอยู่ S1 ไม่ก็ S2

ต่อ ๆ จะ Implement ยังไง

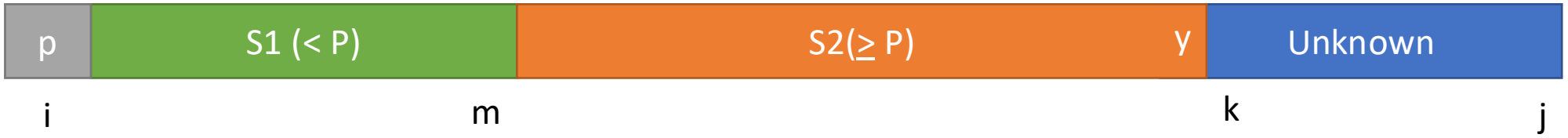
- เริ่มต้น ทั้ง S1 และ S2 ยังว่าง นั่นคือทุกตัวอยู่ใน Unknown
- สำหรับแต่ละสมาชิก a[k] เราก็จะเปรียบเทียบกับ p ว่าจะให้ไปอยู่ไหน
  - ถ้า a[k] ≥ p ก็ใส่ a[k] ไว้ใน S2
  - ถ้าไม่ใช่ ก็ใส่ a[k] ไว้ใน S1
- ถ้าเราพิจารณาครบทุกตัวแล้ว เราก็สลับข้อมูล ตัวสุดท้ายของกลุ่ม S1 กับ pivot เพื่อให้ pivot อยู่ถูกตำแหน่ง

การเอาไปใส่แต่ละกลุ่ม

- กรณีแรก a[k] = y ≥ p

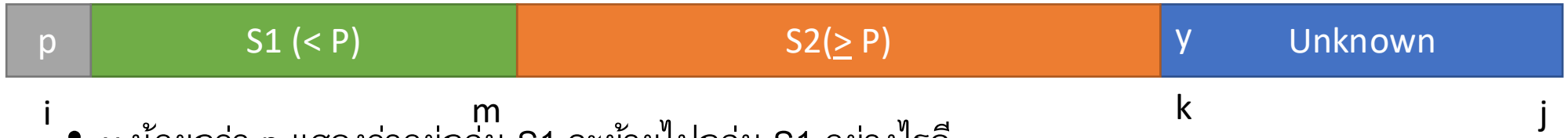


  - y มากกว่าเท่ากับ p แสดงว่าอยู่กลุ่ม S2
  - ดังนั้นเราก็เพิ่มค่า k ที่เป็นตัวแรกของ Unknown ก็พอ

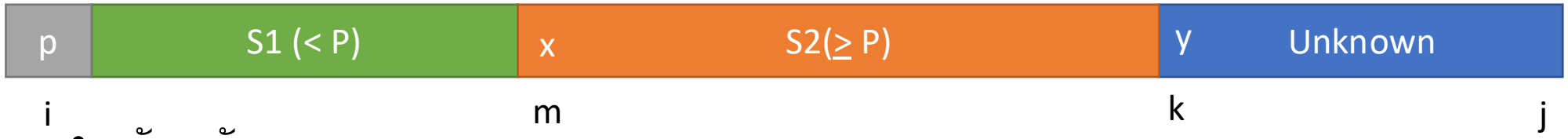


การเอาไปใส่แต่ละกลุ่ม

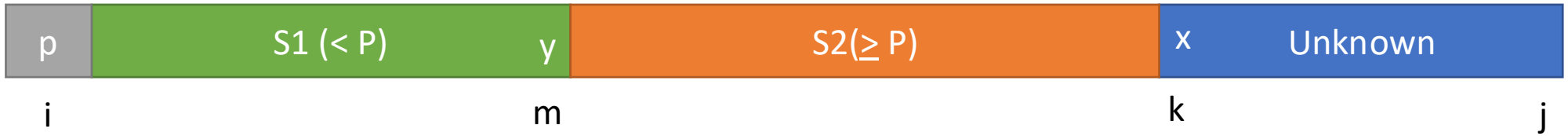
- กรณีที่สอง a[k] = y < p



  - y น้อยกว่า p แสดงว่าอยู่กลุ่ม S1 จะย้ายไปกลุ่ม S1 อย่างไรดี
  - เราก็เพิ่มค่า m ไป 1 สมมติว่า a[mใหม่] = x



  - สลับ x กับ y



  - เพิ่มค่า k

มาเขียน partition กัน

- สิ่งที่รับเข้าฟังก์ชันคือ
  - Array ที่ต้องการแบ่ง
  - i จุดเริ่มต้น ขอบซ้ายของ array
  - j จุดสิ้นสุด ขอบขวาของ array

- สิ่งที่ต้องคืนค่าคือ
  - m คือตำแหน่งที่ถูกต้องของ pivot ใน array ที่เรียงแล้ว

37	48	22	49	37	6

เริ่มต้นให้ p = A[0] = 37

37	48	22	49	37	6

A[1]=48 เป็นส่วน S2 ดังนั้น S1={},S2={48}

37	22	48	49	37	6

A[2]=22 เป็นส่วน S1
สลับ A[1] กับ A[2]
ดังนั้น S1={22},S2={48}

37	22	48	49	37	6

A[3]=49 เป็นส่วน S2 ดังนั้น S1={22},S2={48,49}

37	22	48	49	37	6

A[4]=37 เป็นส่วน S2 ดังนั้น S1={22},S2={48,49,37}

37	22	6	49	37	48

A[5]=6 เป็นส่วน S1
สลับ A[2] กับ A[5]
ดังนั้น S1={22,6},S2={49,37,48}

6	22	37	49	37	48

สลับ A[0] กับ A[2]
ดังนั้น S1={6,22},p={37},S2={49,37,48}

ลองเขียนใน elab

https://cmu.to/partition

```
int partition(int a[], int i, int j) {
  int p = a[i]; // p is the pivot
  int m = i; // S1 and S2 are initially empty
 // เลื่อน k มาพิจารณาทีละตัว




  swap(a[i], a[m]); // final step, swap pivot with a[m]
  return m; // return the index of pivot
}
```

ลองเขียนใน elab

```
int partition(int a[], int i, int j) {
  int p = a[i]; // p is the pivot
  int m = i; // S1 and S2 are initially empty
  for (int k = i+1; k <= j; k++) { // explore the unknown region
    if (a[k] < p) { // case 2
      m++;
      swap(a[k], a[m]); // C++ STL algorithm std::swap
    } // notice that we do nothing in case 1: a[k] >= p
  }
  swap(a[i], a[m]); // final step, swap pivot with a[m]
  return m; // return the index of pivot
}
```